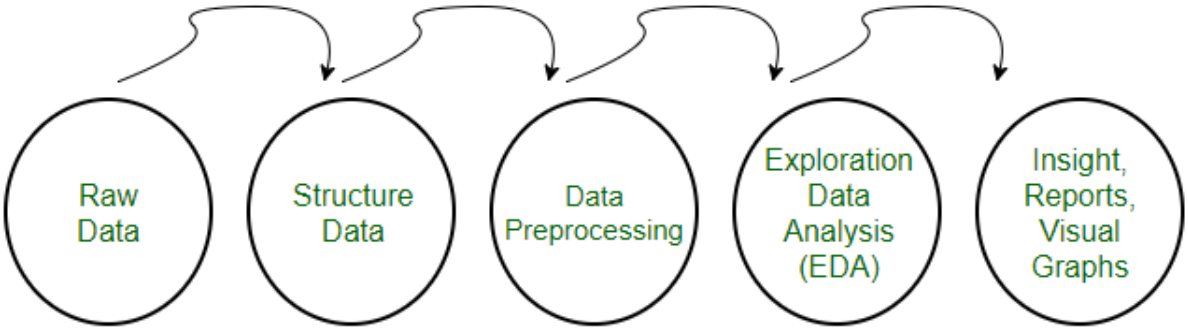
# Data Preprocessing

Pre-processing refers to the transformations applied to our data before feeding it to the algorithm. Data Preprocessing is a technique that is used to convert the raw data into a clean data set. In other words, whenever the data is gathered from different sources it is collected in raw format which is not feasible for the analysis.




## Why do we need Data Preprocessing?

A real-world data generally contains noises, missing values, and maybe in an unusable format which cannot be directly used for machine learning models. Data preprocessing is required tasks for cleaning the data and making it suitable for a machine learning model which also increases the accuracy and efficiency of a machine learning model.

# Data cleaning

## I. Assess Data quality

The term “data quality” refers to the suitability of data to serve its intended purpose. So, measuring data quality involves performing data quality assessments to determine the degree to which your data adequately supports the business needs of the company.

A data quality assessment is done by measuring particular features of the data to see if they meet defined standards. Each such feature is called a “data quality dimension” and is rated according to a relevant metric that provides an objective assessment of quality.

The industry hasn’t yet settled on a standard set of data quality dimensions, but the following is a representative group:

**Four metrics of data quality**

Let's take a brief look at each of these and at the metrics used in assessing them.

**1. Completeness**

Completeness relates to whether all required information is present in the dataset. For example, if the customer information in a database is required to include both first and last names, any record in which the first name or last name field is not populated is marked as incomplete. The metric used in assessing this dimension is the percentage of records that are complete.

**2. Validity**

Data is characterized as valid if it matches the rules specified for it. Those rules typically include specifications such as format (number of digits, etc.), allowable types (integer, floating-point, string, etc.), and range (minimum and maximum values). For example, a telephone number field that contains the string '1809 Oak Street' is not valid. The metric for this dimension is the percentage of records in which all values are valid.

**3. Timeliness**

Timeliness relates to whether the information is up to date for the intended use. In other words, is the correct information available when needed?

For example, if a customer has notified the company of an address change, but the new address is not in the database at the time billing statements are processed, that entry fails the timeliness test. The metric used to measure timeliness is the time difference between when data is needed and when it is available.

**4. Consistency**

A data item is consistent if all representations of that item across data stores match.

If, for example, a birth date is entered in one system using the U.S. format (mm/dd/yyyy), but it is imported into another system where the date is entered using the European standard (dd/mm/yyyy), that data lacks consistency.

## II. Data Anomalies

Generally speaking, an anomaly is something that differs from a norm: a deviation, an exception. In software engineering, by anomaly we understand a rare occurrence or event that doesn't fit into the pattern, and, therefore, seems suspicious. Some examples are:

- sudden burst or decrease in activity
- error in the text;
- sudden rapid drop or increase in temperature.

Common reasons for outliers are:

- data preprocessing errors
- noise
- fraud
- attacks

## III. Detect missing values with pandas dataframe functions: .info() and .isna()

### Pandas DataFrame info() Method

The info() method prints information about the DataFrame. The information contains the number of columns, column labels, column data types, memory usage, range index, and the number of cells in each column (non-null values).

***Syntax***

*dataframe*.info(verbose,buf,max_cols,memory_usage,show_counts,null_counts)

***Return Value***

None. The info() method does not return any value, it prints the information.

```
import pandas as pd
df = pd.read_csv('data.csv')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 169 entries, 0 to 168
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Duration  169 non-null    int64
 1   Pulse     169 non-null    int64
 2   Maxpulse  169 non-null    int64
 3   Calories  164 non-null    float64
```

```
dtypes: float64(1), int64(3)
memory usage: 5.4 KB
```

**Python | Pandas dataframe.isna()**

Python is a great language for doing data analysis, primarily because of the fantastic ecosystem of data-centric python packages. ***Pandas*** is one of those packages and makes importing and analyzing data much easier.

Pandas **dataframe.isna()** function is used to detect missing values. It return a boolean same-sized object indicating if the values are NA. NA values, such as None or numpy. NaN, gets mapped to True values. Everything else gets mapped to False values. Characters such as empty strings ” or numpy.inf are not considered NA values (unless you set pandas.options.mode.use_inf_as_na = True).

***Syntax:*** *DataFrame.isna()*
***Returns :*** *Mask of bool values for each element in DataFrame that indicates whether an element is not an NA value.*

**Example #1:** Use isna() function to detect the missing values in a dataframe

```
# importing pandas as pd
import pandas as pd
# Creating the dataframe
df = pd.read_csv("nba.csv")
# Print the dataframe
df
```

| | Name | Team | Number | Position | Age | Height | Weight | College | Salary |
|---|---|---|---|---|---|---|---|---|---|
| 0 | Avery Bradley | Boston Celtics | 0.0 | PG | 25.0 | 6-2 | 180.0 | Texas | 7730337.0 |
| 1 | Jae Crowder | Boston Celtics | 99.0 | SF | 25.0 | 6-6 | 235.0 | Marquette | 6796117.0 |
| 2 | John Holland | Boston Celtics | 30.0 | SG | 27.0 | 6-5 | 205.0 | Boston University | NaN |
| 3 | R.J. Hunter | Boston Celtics | 28.0 | SG | 22.0 | 6-5 | 185.0 | Georgia State | 1148640.0 |
| 4 | Jonas Jerebko | Boston Celtics | 8.0 | PF | 29.0 | 6-10 | 231.0 | NaN | 5000000.0 |
| 5 | Amir Johnson | Boston Celtics | 90.0 | PF | 29.0 | 6-9 | 240.0 | NaN | 12000000.0 |
| 6 | Jordan Mickey | Boston Celtics | 55.0 | PF | 21.0 | 6-8 | 235.0 | LSU | 1170960.0 |
| 7 | Kelly Olynyk | Boston Celtics | 41.0 | C | 25.0 | 7-0 | 238.0 | Gonzaga | 2165160.0 |
| 8 | Terry Rozier | Boston Celtics | 12.0 | PG | 22.0 | 6-2 | 190.0 | Louisville | 1824360.0 |
| 9 | Marcus Smart | Boston Celtics | 36.0 | PG | 22.0 | 6-4 | 220.0 | Oklahoma State | 3431040.0 |
| 10 | Jared Sullinger | Boston Celtics | 7.0 | C | 24.0 | 6-9 | 260.0 | Ohio State | 2569260.0 |
| 11 | Isaiah Thomas | Boston Celtics | 4.0 | PG | 27.0 | 5-9 | 185.0 | Washington | 6912869.0 |
| 12 | Evan Turner | Boston Celtics | 11.0 | SG | 27.0 | 6-7 | 220.0 | Ohio State | 3425510.0 |
| 13 | James Young | Boston Celtics | 13.0 | SG | 20.0 | 6-6 | 215.0 | Kentucky | 1749840.0 |
| 14 | Tyler Zeller | Boston Celtics | 44.0 | C | 26.0 | 7-0 | 253.0 | North Carolina | 2616975.0 |
| 15 | Bojan Bogdanovic | Brooklyn Nets | 44.0 | SG | 27.0 | 6-8 | 216.0 | NaN | 3425510.0 |
| 16 | Markel Brown | Brooklyn Nets | 22.0 | SG | 24.0 | 6-3 | 190.0 | Oklahoma State | 845059.0 |

```
# detect the missing values
df.isna()
```

| | Name | Team | Number | Position | Age | Height | Weight | College | Salary |
|---|---|---|---|---|---|---|---|---|---|
| 0 | False | False | False | False | False | False | False | False | False |
| 1 | False | False | False | False | False | False | False | False | False |
| 2 | False | False | False | False | False | False | False | False | True |
| 3 | False | False | False | False | False | False | False | False | False |
| 4 | False | False | False | False | False | False | False | True | False |
| 5 | False | False | False | False | False | False | False | True | False |
| 6 | False | False | False | False | False | False | False | False | False |
| 7 | False | False | False | False | False | False | False | False | False |
| 8 | False | False | False | False | False | False | False | False | False |
| 9 | False | False | False | False | False | False | False | False | False |
| 10 | False | False | False | False | False | False | False | False | False |
| 11 | False | False | False | False | False | False | False | False | False |
| 12 | False | False | False | False | False | False | False | False | False |
| 13 | False | False | False | False | False | False | False | False | False |
| 14 | False | False | False | False | False | False | False | False | False |
| 15 | False | False | False | False | False | False | False | True | False |
| 16 | False | False | False | False | False | False | False | False | False |
| 17 | False | False | False | False | False | False | False | False | False |

**Example #2:** Use `isna()` function to detect missing values in a pandas series object

```
# importing pandas as pd
import pandas as pd
# Creating the series
sr = pd.Series([12, 5, None, 5, None, 11])
# Print the series
sr
```

```
0    12.0
1     5.0
2     NaN
3     5.0
4     NaN
5    11.0
dtype: float64
```

```
# to detect the missing values
sr.isna()
```

```
0    False
1    False
2     True
3    False
4     True
5    False
dtype: bool
```

## Approaches to deal with missing values

**Keep the missing value as is**
Sometimes missing data is very less number of rows (say less than 3%) then we can simply ignore the missing data. There is no hard rule to keep the missing data it depends on us

**Remove data objects with missing values (Deleting the entire column)**

If a certain column has many missing values, then you can choose to drop the entire column.

Code to drop the entire column is as follows:

```
IN:

df = train_df.drop(['Dependents'],axis=1)

df.isnull().sum()


OUT:

Loan_ID  0

Gender  13

Married  3

Education  0

Self_Employed 32

ApplicantIncome  0

CoapplicantIncome  0

LoanAmount  22
```

**Remove the attributes with missing values (Deleting the entire row)**
If a row has many missing values, then you can choose to drop the entire row.
If every row has some (column) value missing, then you might end up deleting the whole data.
Code to drop the entire row is as follows:

```
IN:

df = train_df.dropna(axis=0)

df.isnull().sum()

 OUT:

Loan_ID  0

Gender  0

Married  0

Dependents  0

Education  0

Self_Employed 0
```

```
ApplicantIncome  0
CoapplicantIncome  0
LoanAmount  0
Loan_Amount_Term  0
Credit_History  0
Property_Area  0
Loan_Status  0
dtype: int64
```

**Estimate and impute missing values**

**Replacing with Arbitrary Value**

If you can make an educated guess about the missing value, then you can replace it with some arbitrary value using the following code.

Ex. In the following code, we are replacing the missing values of the 'Dependents' column with '0'

IN:

```
#Replace the missing value with '0' using 'fiilna' method
train_df['Dependents'] = train_df['Dependents'].fillna(0)
train_df['Dependents'].isnull().sum()
```

OUT:

```
0
```

**Replacing With Mean**

This is the most common method of imputing missing values of numeric columns. If there are outliers, then the mean will not be appropriate. In such cases, outliers need to be treated first. You can use the 'fillna' method for imputing the columns 'LoanAmount' and 'Credit_History' with the mean of the respective column values.

**IN:**

```
#Replace the missing values for numerical columns with mean
train_df['LoanAmount'] = train_df['LoanAmount'].fillna(train_df['LoanAmount'].mean())
train_df['Credit_History'] =
train_df['Credit_History'].fillna(train_df['Credit_History'].mean())
```

**OUT:**

```
Loan_ID  0
```

**Replacing with Mode**

Mode is the most frequently occurring value. It is used in the case of categorical features.
You can use the 'fillna' method for imputing the categorical columns 'Gender', 'Married', and 'Self_Employed'.

**IN:**

```
#Replace the missing values for categorical columns with mode

train_df['Gender'] = train_df['Gender'].fillna(train_df['Gender'].mode()[0])

train_df['Married'] = train_df['Married'].fillna(train_df['Married'].mode()[0])

train_df['Self_Employed'] =
train_df['‘Self_Employed'].fillna(train_df['Self_Employed'].mode()[0])

train_df.isnull().sum()
```

**Replacing with Median**

Median is the middlemost value. It's better to use the median value for imputation in the case of outliers.
You can use 'fillna' method for imputing the column 'Loan_Amount_Term' with the median value.

```
train_df['Loan_Amount_Term']=
train_df['Loan_Amount_Term'].fillna(train_df['Loan_Amount_Term'].m
```

**Replacing with previous value – Forward fill**

In some cases, imputing the values with the previous value instead of mean, mode or median is more appropriate. This is called forward fill. It is mostly used in time series data.
You can use 'fillna' function with the parameter 'method = ffill'

IN:

```
import pandas as pd

import numpy as np

test = pd.Series(range(6))

test.loc[2:4] = np.nan

test
```

**Replacing with next value – Backward fill**

In backward fill, the missing value is imputed using the next value.

**IN:**

```
# Backward-Fill

test.fillna(method='bfill')
```